

Part 1: Environment Setup & Mini Project

Group Members

- Ahammad, Jony
- Kumara, Muditha
- Podkorytov, Nikolai
- Shrestha Bata, Prabesh

1. Framework Choice & Installation

Which Framework Was Selected

Framework: TensorFlow and PyTorch both

Installation Method

Method Used: pip

Commands Executed:

```
pip install tensorflow  
pip install torch torchvision
```

Installation Status:  Successful

```
(myenv) muditha@lap:~/Applied-artificial-intelligence$ python3 --version
Python 3.10.12
(myenv) muditha@lap:~/Applied-artificial-intelligence$ python -c "import tensorflow as tf; print(tf.__version__)"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776207136.647845 45154 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
I0000 00:00:1776207136.649956 45154 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
I0000 00:00:1776207136.926408 45154 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776207138.326102 45154 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
I0000 00:00:1776207138.327748 45154 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
2.21.0
(myenv) muditha@lap:~/Applied-artificial-intelligence$ python -c "import torch; print(torch.__version__)"
2.11.0+cu130
```

2. Virtual Environment Setup

Environment Configuration

Virtual Environment Tool: venv

Setup Commands:

```
python3 -m venv myenv
source myenv/bin/activate
```

```
(myenv) muditha@lap:~/Applied-artificial-intelligence$
```

Environment Details

- **Python Version:** 3.10.12
- **Framework Version:** TensorFlow 2.21.0, PyTorch 2.11.0+cu130

```
(myenv) muditha@lap:~/Applied-artificial-intelligence$ python3 --version
Python 3.10.12
(myenv) muditha@lap:~/Applied-artificial-intelligence$ python -c "import tensorflow as tf; print(tf.__version__)"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776207136.647845 45154 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
I0000 00:00:1776207136.649956 45154 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
I0000 00:00:1776207136.926408 45154 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776207138.326102 45154 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
I0000 00:00:1776207138.327748 45154 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
2.21.0
(myenv) muditha@lap:~/Applied-artificial-intelligence$ python -c "import torch; print(torch.__version__)"
2.11.0+cu130
```

- **Dependencies Installed:**

```
(myenv) muditha@lap:~/Applied-artificial-intelligence$ pip list
```

Package	Version
-----	-----
absl-py	2.4.0
astunparse	1.6.3
certifi	2026.2.25
charset-normalizer	3.4.7
cuda-bindings	13.2.0
cuda-pathfinder	1.5.3
cuda-toolkit	13.0.2
filelock	3.25.2
flatbuffers	25.12.19
fsspec	2026.3.0
gast	0.7.0
google-pasta	0.2.0
grpcio	1.80.0
h5py	3.14.0
idna	3.11
Jinja2	3.1.6
keras	3.12.1
libclang	18.1.1
markdown-it-py	4.0.0
MarkupSafe	3.0.3
mdurl	0.1.2
ml_dtypes	0.5.4
mpmath	1.3.0
namex	0.1.0
networkx	3.4.2
numpy	2.2.6
nvidia-cublas	13.1.0.3
nvidia-cuda-cupti	13.0.85
nvidia-cuda-nvrtc	13.0.88
nvidia-cuda-runtime	13.0.96
nvidia-cudnn-cu13	9.19.0.56
nvidia-cufft	12.0.0.61
nvidia-cufile	1.15.1.6
nvidia-curand	10.4.0.35
nvidia-cusolver	12.0.4.66
nvidia-cusparse	12.6.3.3
nvidia-cusparselt-cu13	0.8.0
nvidia-nccl-cu13	2.28.9
nvidia-nvjitlink	13.0.88
nvidia-nvshmem-cu13	3.4.5
nvidia-nvtx	13.0.85
opt_einsum	3.4.0
optree	0.19.0
packaging	26.1

pillow	12.2.0
pip	26.0.1
protobuf	7.34.1
Pygments	2.20.0
requests	2.33.1
rich	15.0.0
setuptools	59.6.0
six	1.17.0
sympy	1.14.0
tensorflow	2.21.0
termcolor	3.3.0
torch	2.11.0
torchaudio	2.11.0
torchvision	0.26.0
triton	3.6.0
typing_extensions	4.15.0
urllib3	2.6.3
wheel	0.46.3
wrapt	2.1.2

• (myenv) muditha@lap:~/Applied-artificial-intelligence\$

3. GPU Verification & Configuration

GPU Detection Code

```
"""GPU detection script for TensorFlow and PyTorch."""

def check_tensorflow_gpu():
    try:
        import tensorflow as tf

        gpus = tf.config.list_physical_devices('GPU')
        print('TensorFlow GPUs:', gpus)
        print('TensorFlow GPU count:', len(gpus))
    except Exception as e:
        print('TensorFlow error:', e)

def check_pytorch_gpu():
    try:
        import torch

        print('PyTorch CUDA available:', torch.cuda.is_available())
        print('PyTorch CUDA version:', torch.version.cuda)
        print('PyTorch GPU count:', torch.cuda.device_count())

        if torch.cuda.is_available():
            for i in range(torch.cuda.device_count()):
                print(f'GPU {i}:', torch.cuda.get_device_name(i))
    except Exception as e:
        print('PyTorch error:', e)

if __name__ == '__main__':
    print('=' * 50)
    print('GPU DETECTION')
    print('=' * 50)
```

```
check_tensorflow_gpu()  
print()  
check_pytorch_gpu()
```

Output Screenshot:

```
(cv_project) muditha@lap:~/Applied-artificial-intelligence/3.2$ python3 gpu_detection.py  
===== GPU DETECTION =====  
TensorFlow error: No module named 'tensorflow'  
  
PyTorch CUDA available: True  
PyTorch CUDA version: 13.0  
PyTorch GPU count: 1  
/home/muditha/VisionProject/cv_project/lib/python3.10/site-packages/torch/cuda/_init_.py:371: UserWarning: Found GPU0 NVIDIA GeForce MX350 which is of compute capability (CC) 6.1.  
The following list shows the CCs this version of PyTorch was built for and the hardware CCs it supports:  
- 7.5 which supports hardware CC >=7.5,<8.0  
- 8.0 which supports hardware CC >=8.0,<9.0 except {8.7}  
- 8.6 which supports hardware CC >=8.6,<9.0 except {8.7}  
- 9.0 which supports hardware CC >=9.0,<10.0  
- 10.0 which supports hardware CC >=10.0,<11.0 except {10.1}  
- 12.0 which supports hardware CC >=12.0,<13.0  
Please follow the instructions at https://pytorch.org/get-started/locally/ to install a PyTorch release that supports one of these CUDA versions: 12.6  
warn_unsupported code(d, device_cc, code_ccs)  
/home/muditha/VisionProject/cv_project/lib/python3.10/site-packages/torch/cuda/_init_.py:489: UserWarning:  
NVIDIA GeForce MX350 with CUDA capability sm_61 is not compatible with the current PyTorch installation.  
The current PyTorch install supports CUDA capabilities sm_75 sm_80 sm_86 sm_90 sm_100 sm_120.  
If you want to use the NVIDIA GeForce MX350 GPU with PyTorch, please check the instructions at https://pytorch.org/get-started/locally/  
  
queued_call()  
GPU 0: NVIDIA GeForce MX350
```

GPU Status

- **GPU Available:**  Yes
- **Number of GPUs Detected:** 1
- **GPU Name(s):** NVIDIA GeForce MX350
- **CUDA Version:** 13.0 (PyTorch build)
- **Compatibility Note:**  GPU compute capability is sm_61, but the installed PyTorch build supports sm_75 and above.

Any Issues Encountered & Resolution

- **Issue 1 (PyTorch GPU compatibility):** PyTorch detected the MX350 GPU, but raised warnings that CUDA capability sm_61 is not compatible with the installed build.
 - **Resolution:** Run PyTorch on CPU in the current setup. Because my GPU is very old.

4. Minimal Test Script

Test Code Implementation

TensorFlow Example

```
import tensorflow as tf
import numpy as np
import os

# Force TensorFlow to use CPU only due to older VGA
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"

# Verify environment isolation and device usage
print("TensorFlow version:", tf.__version__)
print("Available devices:", tf.config.list_physical_devices())

# 1. Improved model definition using Input layer
from tensorflow.keras import layers, models

model = models.Sequential(
    [
        layers.Input(shape=(20,)), # Explicit input layer
        layers.Dense(32, activation="relu"), # Hidden layer
        layers.Dense(2, activation="softmax"), # Output layer
    ]
)

# 2. Compile the model with recommended settings
model.compile(
    optimizer="adam", loss="sparse_categorical_crossentropy", metrics=|

# 3. Model Summary for verification
model.summary()

# 4. Dummy data for testing (10 samples, 20 features)
```

```
data = np.random.rand(10, 20)
```

```
# 5. Execute prediction
```

```
predictions = model.predict(data)
```

```
print("\nPredictions shape:", predictions.shape)
```

```
print("Predictions:\n", predictions)
```


PyTorch Example

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# Force PyTorch to use CPU
device = torch.device("cpu")
print(f"Using device: {device}")
print("PyTorch version:", torch.__version__)

# 1. Define a simple model architecture
class MinimalNet(nn.Module):
    def __init__(self):
        super(MinimalNet, self).__init__()
        self.fc1 = nn.Linear(20, 32) # Input layer to hidden
        self.fc2 = nn.Linear(32, 2) # Hidden to output

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return F.softmax(x, dim=1)

# 2. Initialize the model and move to CPU
model = MinimalNet().to(device)
print("\nModel Architecture:\n", model)

# 3. Generate dummy data (10 samples, 20 features)
# PyTorch requires data to be in Tensor format
data = torch.randn(10, 20).to(device)

# 4. Run prediction (forward pass)
model.eval() # Set to evaluation mode
with torch.no_grad():
    predictions = model(data)
```

```
print("\nPredictions shape:", predictions.shape)
print("Predictions:\n", predictions)
```

Console Output Screenshot:

```
(myenv) muditha@lap:~/Applied-artificial-intelligence/3.2$ python3 PyTorch-Example.py
Using device: cpu
PyTorch version: 2.11.0+cu130

Model Architecture:
MinimalNet(
  (fc1): Linear(in_features=20, out_features=32, bias=True)
  (fc2): Linear(in_features=32, out_features=2, bias=True)
)

Predictions shape: torch.Size([10, 2])
Predictions:
tensor([[0.5986, 0.4014],
        [0.5701, 0.4299],
        [0.2736, 0.7264],
        [0.4938, 0.5062],
        [0.3357, 0.6643],
        [0.4840, 0.5160],
        [0.4534, 0.5466],
        [0.3610, 0.6390],
        [0.4704, 0.5296],
        [0.3701, 0.6299]])

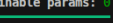
(myenv) muditha@lap:~/Applied-artificial-intelligence/3.2$
```

```
(myenv) muditha@lap:~/Applied-artificial-intelligence/3.2$ python3 Tensor-Keras-Example.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776352542.878027 27058 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation o
rders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
I0000 00:00:1776352542.878279 27058 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
I0000 00:00:1776352542.913950 27058 cpu feature guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1776352543.869246 27058 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation o
rders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
I0000 00:00:1776352543.869757 27058 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
TensorFlow version: 2.21.0
E0000 00:00:1776352543.979506 27058 cuda_platform.cc:52] failed call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: CUDA_ERROR_NO_DEVICE: no CUDA-capable device is detected
I0000 00:00:1776352543.979532 27058 cuda_diagnostics.cc:160] env: CUDA_VISIBLE_DEVICES=""
I0000 00:00:1776352543.979555 27058 cuda_diagnostics.cc:163] CUDA_VISIBLE_DEVICES is set to -1 - this hides all GPUs from CUDA
I0000 00:00:1776352543.979563 27058 cuda_diagnostics.cc:171] verbose logging is disabled. Rerun with verbose logging (usually --v=1 or --vmodule=cuda_diagnostics=1) to get more diagnostic output fro
m this module
I0000 00:00:1776352543.979565 27058 cuda_diagnostics.cc:176] retrieving CUDA diagnostic information for host: lap
I0000 00:00:1776352543.979567 27058 cuda_diagnostics.cc:183] hostname: lap
I0000 00:00:1776352543.979613 27058 cuda_diagnostics.cc:190] libcuda reported version is: 580.126.9
I0000 00:00:1776352543.979623 27058 cuda_diagnostics.cc:194] kernel reported version is: 580.126.9
I0000 00:00:1776352543.979626 27058 cuda_diagnostics.cc:284] kernel version seems to match DSO: 580.126.9
Available devices: [PhysicalDevice(name='/physical_device:CPU:0', device_type='CPU')]
Model: "sequential"



| Layer (type)    | Output Shape | Param # |
|-----------------|--------------|---------|
| dense (Dense)   | (None, 32)   | 672     |
| dense_1 (Dense) | (None, 2)    | 66      |



Total params: 738 (2.88 KB)
Trainable params: 738 (2.88 KB)
Non-trainable params: 0 (0.00 B)
1/1  0s 59ms/step

Predictions shape: (10, 2)
Predictions:
[[0.32536435 0.6746356 ]
 [0.17509975 0.82490027]
 [0.20145491 0.7985451 ]
 [0.35178316 0.64821684]
 [0.17447615 0.8255238 ]
 [0.20881395 0.7991061 ]
 [0.23984487 0.76095515]
 [0.2128702 0.7871298 ]
 [0.2091581 0.79084986]
 [0.25082108 0.7491789 ]]

(myenv) muditha@lap:~/Applied-artificial-intelligence/3.2$
```

Test Results Summary:

- ✔ Model creation: Success

-  Predictions generated: Success
-  GPU utilized: No
-  No errors: No

5. Challenges & Solutions

Challenge 1

Problem: Upon running the initial test scripts, the system encountered multiple `CUDA_ERROR_NO_DEVICE` errors and warnings indicating that CUDA drivers could not be found. Because the local machine uses an older VGA/GPU that does not support the required CUDA versions for TensorFlow 2.21.0, the framework was unable to initialize hardware acceleration.

Solution: The issue was resolved by forcing the deep learning frameworks to operate exclusively on the CPU. For TensorFlow, this was achieved by setting the environment variable `os.environ['CUDA_VISIBLE_DEVICES'] = '-1'`, which effectively hides any incompatible GPU from the library and prevents initialization errors. Additionally, the code was updated to use a dedicated layers.Input object to resolve UserWarnings regarding the model's architecture definition style. This ensured a stable, reproducible environment suitable for completing the assignment without specialized hardware.

7. Environment Documentation

requirements.txt

```
absl-py==2.4.0
astunparse==1.6.3
certifi==2026.2.25
charset-normalizer==3.4.7
cuda-bindings==13.2.0
cuda-pathfinder==1.5.3
cuda-toolkit==13.0.2
filelock==3.25.2
flatbuffers==25.12.19
fsspec==2026.3.0
gast==0.7.0
google-pasta==0.2.0
grpcio==1.80.0
h5py==3.14.0
idna==3.11
Jinja2==3.1.6
keras==3.12.1
libclang==18.1.1
markdown-it-py==4.0.0
MarkupSafe==3.0.3
mdurl==0.1.2
ml_dtypes==0.5.4
mpmath==1.3.0
namex==0.1.0
networkx==3.4.2
numpy==2.2.6
nvidia-cublas==13.1.0.3
nvidia-cuda-cupti==13.0.85
nvidia-cuda-nvrtc==13.0.88
nvidia-cuda-runtime==13.0.96
nvidia-cudnn-cu13==9.19.0.56
nvidia-cufft==12.0.0.61
nvidia-cufile==1.15.1.6
```

```
nvidia-curand==10.4.0.35
nvidia-cusolver==12.0.4.66
nvidia-cusparselt==12.6.3.3
nvidia-cusparse==12.6.3.3
nvidia-cusparselt-cu13==0.8.0
nvidia-nccl-cu13==2.28.9
nvidia-nvjitlink==13.0.88
nvidia-nvshmem-cu13==3.4.5
nvidia-nvtx==13.0.85
opt_einsum==3.4.0
optree==0.19.0
packaging==26.1
pillow==12.2.0
protobuf==7.34.1
Pygments==2.20.0
requests==2.33.1
rich==15.0.0
six==1.17.0
sympy==1.14.0
tensorflow==2.21.0
termcolor==3.3.0
torch==2.11.0
torchaudio==2.11.0
torchvision==0.26.0
triton==3.6.0
typing_extensions==4.15.0
urllib3==2.6.3
wrapt==2.1.2
```

How to reproduce this environment:

```
# Step-by-step commands
```

1. `python -m venv myenv`
2. `source myenv/bin/activate` # On Windows: `myenv\Scripts\activate`
3. `pip install -r requirements.txt`

Appendix: Code Repository

GitHub/Repository Link: <https://github.com/Muditha-Kumara/Applied-artificial-intelligence/tree/main/3.2>

Git Commit: 88ac712722c30c30f8a4d07873bc983a0b28acb1